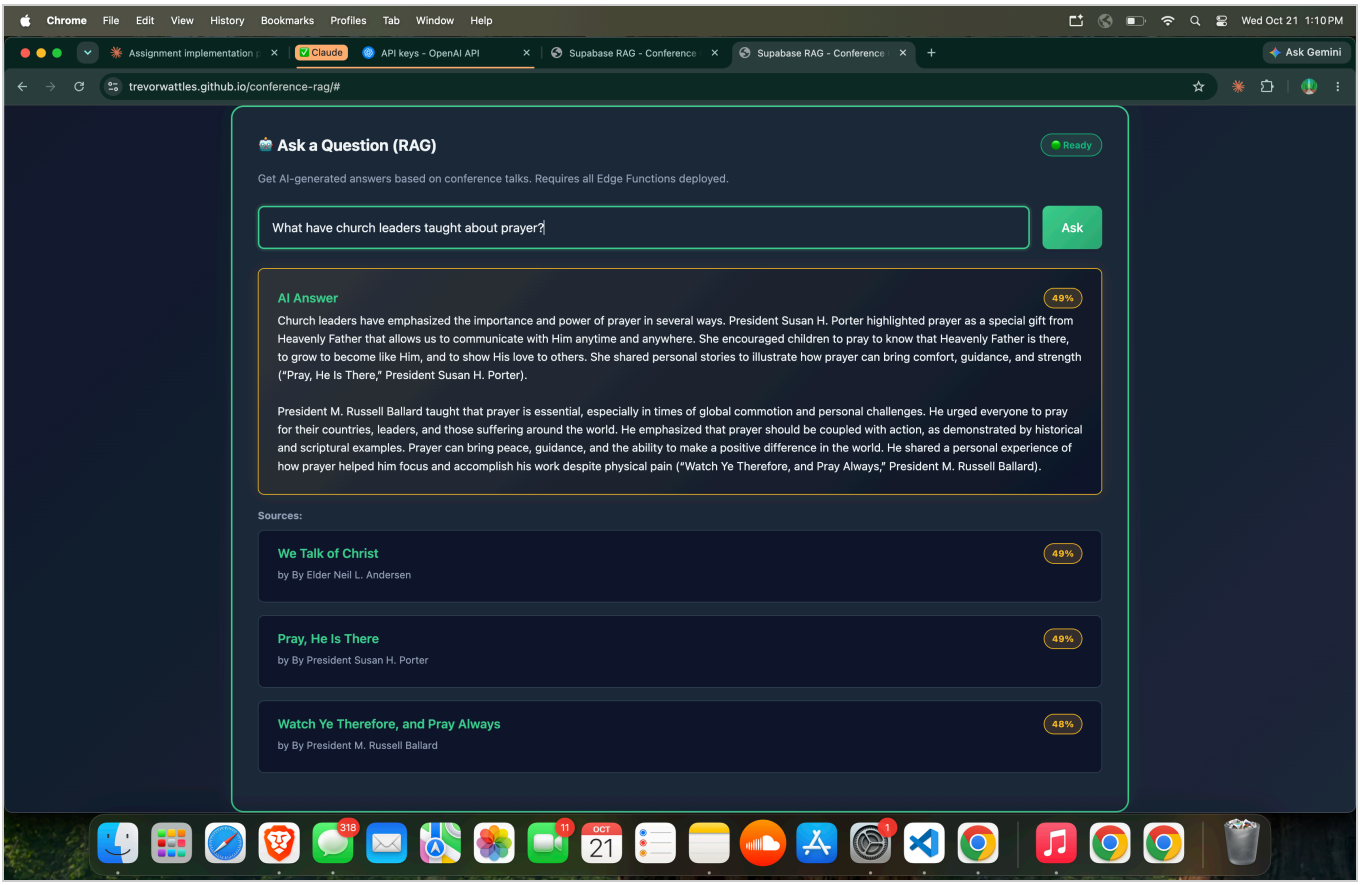
# CS 452: Conference Talk RAG

**Trevor Wattles** · October 21, 2026

| Deliverable | Link / detail |
|---|---|
| **Repository (fork)** | https://github.com/trevorwattles/conference-rag |
| **Live site** | https://trevorwattles.github.io/conference-rag/ |
| **Data** | 426 General Conference talks (April 2020 – October 2025), 26,256 sentence chunks, 1,536-dim embeddings |
| **Tests** | `scripts/test_rls.py` 29/29 · `scripts/test_functions.py` 19/19 (full output in the appendix) |
| **Design & scaling** | `ARCHITECTURE.md` |

## Working RAG query

All three search modes are green. Question: "*What have church leaders taught about prayer?*" The answer cites the talks it used, and each source links to the original talk.



## What I built beyond the README

The professor asked for correct access control using RLS and a design that can scale to millions. These are the changes I made for that, each as a committed migration in `supabase/migrations/`:

- **Grants + RLS, not RLS alone (`0002`).** Supabase grants `ALL` (including `TRUNCATE`, which RLS doesn't cover) to the public `anon` role by default. I revoked that and granted only what each role needs. `match_sentences()` is now `SECURITY INVOKER` with a pinned `search_path`, authenticated-only, capped at 50 rows, and returns `url` (fixing the citation links).
- `page_views` **made write-only.** Before, anyone could read every visitor's user agent. Now clients can insert only `page_url`/`user_agent` (length-checked), and the counter uses a count-only `SECURITY DEFINER` function. I also found that the original code saved the full URL, which right after a magic-link login contains the user's `#access_token`, into that world-readable table. It now stores origin + path only.
- **Indexes, measured with `EXPLAIN ANALYZE` (`0003`, `0004`).** A trigram GIN index for `ILIKE` keyword search (worst case 91 ms → 0.3 ms) and an HNSW index for vector search. Without HNSW, semantic search took 17–23 s, longer than the 8 s timeout for logged-in users, so it would simply have failed. With it: 3–8 ms warm, 90–100% recall versus the exact answer.
- **Edge Functions as the "secure middle."** Auth is verified inside each function. Input is validated before any OpenAI call. CORS allows only my site. `context_talks` from the browser is treated as untrusted (≤5 talks, ≤60k chars).
- **Data cleanup (`0005`)** of scraped speaker names ("by By Elder…").

## My custom feature: Question History + embedding cache + rate limit

**What it does.** A 📜 **My Question History** panel lists every question you've asked. Click one and its answer and sources come back instantly, with no new OpenAI call. Each entry has a delete button. Behind it:

1. `question_history` **with real per-user RLS.** SELECT/INSERT/DELETE policies are all `user_id = (select auth.uid())`, and there's no UPDATE. The frontend never filters by user; the database does. Column-level INSERT grants mean a user can't set `user_id` (to write into someone else's history) or `created_at` (to backdate). `generate-answer` writes the row **using the caller's own JWT**, so RLS still applies even inside my server code.
2. **Embedding cache.** `embed-question` hashes the normalized question (sha256) and reuses stored embeddings, so repeated questions cost $0. The cache table has RLS on with no policies or grants: only the Edge Function (service role) can touch it, and it stores no user IDs.
3. **Per-user rate limit.** 30 answers/hour and 100 new embeddings/hour, enforced by an atomic SQL function over a server-only `api_usage` table.

**Why I built it.** It's the feature that best exercises what the professor asked for. The rest of the app only uses RLS as "logged in or not"; history needs RLS that separates users from each other, and `test_rls.py` proves user B can't read, update or delete user A's rows. The cache and rate limit are cost controls. At GPT-4o prices one answer costs about $0.03, so at a million users OpenAI cost, not CPU, is what breaks first.

**A design flaw I caught.** My first plan counted `question_history` rows for the rate limit. But users can delete their own history, which would reset the limit, and a script calling the function directly would never

create history rows at all. So the limit lives in a table users can't read, write or delete. A security control is only as strong as the data it counts.

## Reflection questions

### 1. Security architecture




- **Why the anon key is safe to publish.** It isn't a password. It only says "this request is from my app, acting as the `anon` role." Everything that role can do is decided inside the database, and I revoked almost all of it. My test uses only the anon key and gets `permission denied` on the talks, the search function and every write. To read the talks you need a JWT with `role: authenticated`, which Supabase signs and only issues after a magic-link login, so it can't be forged.
- **If someone got the service_role key.** That key bypasses RLS. They could read, change or delete every table, including every user's question history. They could use the Auth admin API to create or delete users, and run up my Supabase usage. I'd have to rotate it immediately. That's why it lives only in a git-ignored file on my laptop and in the Edge runtime's environment.
- **How Edge Functions protect the OpenAI key.** The key is stored as a function secret and only ever used server-side. The browser sends a question plus its user JWT. The function checks the JWT with Supabase Auth, validates the input, checks the rate limit, and only then calls OpenAI, returning just the result.
- **RLS's role.** It's the database enforcing access per row, so a bug in my frontend (or someone skipping my frontend and calling the API directly) still can't see other people's data. I learned it's the *second* layer: GRANTs decide which operations are allowed at all (`42501 permission denied`), and RLS then filters rows (B asking for A's row just gets 0 rows, as if it doesn't exist).

### 2. Edge Functions and the "secure middle"

**Why not call OpenAI from the browser?** Anything the browser has, the user has. The key would be visible in DevTools or the page source, and anyone could copy it and run up charges on my account with no limits, and I couldn't tell them apart from my real users.

**The flow:** browser → `embed-question` / `generate-answer` → OpenAI. The browser sends `Authorization: Bearer <user JWT>`. The function calls `supabase.auth.getUser(token)`. No token, or just the anon key (a valid JWT with no user behind it), gets **401** before any work is done. Then it validates the input (400 for bad input, still with no OpenAI call), checks the user's rate limit (429), and only then uses its secret key.

**JWT verification's role:** it's what turns "some HTTP request" into "this specific user", which is what makes per-user rate limits and per-user history possible. One detail I learned: my project signs user sessions with **ES256** (asymmetric) while the anon key is **HS256**, and the gateway's built-in check couldn't verify them. So I deploy with `--no-verify-jwt` and verify in code. That makes the function *my* responsibility: if I deleted the `requireAuth` line, it would be open to the internet.

**Other applications:** any third-party secret works this way: Stripe charges, sending email/SMS, a mapping API key, admin-only actions. The browser asks a small server you control, which authenticates the user, enforces business rules and limits, then uses the secret. Often called a "backend for frontend."

### 3. From SQL to semantics

| Query | Keyword ( `ILIKE` ) | Semantic (cosine similarity) |
|---|---|---|
| "*How can I find peace during hard times?*" | **0 results**: no sentence contains that exact string | **Wins.** Top hit (0.63): "There is peace and joy even in hard times." Also: "…you can find true rest—meaning relief and peace", "enduring peace and joy, even during turbulent times." |
| "*xylophone*" (in no talk) | **Wins.** Correctly returns 0 results | Still returns 20 results (organ bellows, a choir melody, a piano). Close in meaning, but never says "not found." |
| "*Which talk mentions Christ the most?*" | **Wins (via SQL).** Exact count over all 426 talks: "And We Talk of Christ" (Elder Gary E. Stevenson), 45 | RAG only sees the top 3 talks, so it confidently picked "We Talk of Christ" (Elder Neil L. Andersen), which is actually **#5** with 33 |

- **When keyword/SQL is better:** exact phrases, names and quotes; when "no results" should be a real answer; counts and "which is the most"; and when you need every match rather than the top 20. It's also cheaper: no embedding call, and with a trigram index it's under a millisecond.
- **When semantic is better:** when people describe an idea in their own words. Users don't know which words a speaker used, and semantic search closes that vocabulary gap ("hard times" ↔ "turbulent times," "rest," "relief").
- **What makes semantic search "understand" meaning:** the embedding model maps text to a point in 1,536-dimensional space, placed so that texts used in similar contexts land near each other. "Peace in hard times" and "rest and relief" end up pointing in similar directions. Cosine similarity measures the angle between two vectors, so the search is "find the sentences pointing most nearly the same way as my question." It's learned statistical similarity, not understanding in a human sense, which is why "xylophone" lands on pianos.

(Surprise: I expected semantic search to fail on a rare proper noun like "Kirtland," but all 20 of its results were about Kirtland. The rare word dominates its embedding.)

## 4. RAG vs fine-tuning

**Fine-tuning** means continuing to train a model on your own examples so its weights change. It's good at teaching *behavior*: a style, a format, a classification task, domain vocabulary. It's unreliable for teaching *facts*: the model absorbs them fuzzily, can't point to where a fact came from, and still hallucinates.

| | **RAG (what I built)** | **Fine-tuning** |
|---|---|---|
| New content (e.g. April 2026 conference) | Scrape + embed new talks: minutes, about 2 cents (all 26k chunks cost $0.018) | Retrain the model, then re-evaluate it |
| Sources and citations | Built in: we know exactly which talks were sent, and the UI links them | No: knowledge is mixed into the weights |
| Hallucination control | Grounded in retrieved text. Asked "What is the capital of France?" with only a prayer talk as context, it answered "The excerpts provided do not contain information about the capital of France." | The model answers from memory either way |
| Cost and effort | Embeddings + a vector index + a prompt | Training data prep, training runs, hosting a custom model |
| Weak spots | Only as good as retrieval (top-k misses aggregate questions, see #3), and limited by the context window | Style and behavior can be very consistent; no retrieval step |

**Why RAG here:** the content changes every six months, users care about *which talk* said something, and accuracy to the source matters more than style.

**When fine-tuning is better:** when you need consistent behavior rather than facts, such as always answering in a specific format or voice, classifying text, or using specialized jargon correctly. Also when a small fine-tuned model can replace a big general one to cut cost and latency at scale. The two combine well: fine-tune for behavior, and use RAG for the facts.

## 5. AI-assisted development

I used **Claude Code** (Anthropic's coding agent) in VS Code for the whole project, and gave it my GitHub CLI login and a Supabase access token. I also had it draft these written answers from our session, then reviewed them.

- **What it did well.** It moved fast because it had real access: it forked the repo, enabled Pages, set auth URLs through the Management API, ran migrations and deployed functions from the terminal. It handled secrets carefully: keys were read from files inside scripts, never printed into the chat, and it checked `git status` before every commit. It also caught problems I wouldn't have known to look for: Supabase's default `ALL` grants, the access token that would have been saved into a public table, a rate-limit design users could reset, and semantic search being slower than the logged-in timeout until the vector index existed. And it measured before and after instead of just claiming things were faster.

- **Where I had to guide it.** I made the product decisions. I chose the custom feature, and I tested every milestone myself on the live site, including the screenshots, which is how the "by By Elder…" speaker bug and an invisible delete button got caught. I also stopped one of its commands partway through the vector-index work. It was confidently wrong at times: it guessed "cold cache" for slow searches before buffer statistics showed its own measurement script was evicting the index, it followed a plan tip (more memory for the index build) that made the free-tier build fail, and a couple of the course's commands and scripts didn't match current tool versions. In each case the fix came from checking real output, not from accepting the first explanation.
- **What I learned about working with AI tools.** Giving an agent credentials is a big speedup, but it raises the stakes. Use short-lived, least-privilege tokens (I created my Supabase access token with a short expiry), keep secrets out of anything that gets shared (this transcript gets uploaded), and rotate anything that leaks. Ask it to prove claims with a test or an `EXPLAIN` rather than trusting its explanation. Its first theory is a hypothesis. And it's only as good as its review: the security improvements came from it questioning the original design, but the bugs it couldn't see came from me looking at the actual site.

---

## Appendix A: RLS proof test (`python scripts/test_rls.py`)

```
============================================================
RLS Proof Test (all checks use the public anon key)
============================================================

👤 Anonymous visitor (anon key, no login)
  ✅ can insert a page view
       → ok (inserted)
  ✅ can call page_view_count()
       → ok (21)
  🔒 cannot read page_views rows (visitor user agents)
       → denied: 42501 permission denied for table page_views
  🔒 cannot forge visited_at on a page view
       → denied: 42501 permission denied for table page_views
  🔒 cannot insert an oversized page_url (CHECK constraint)
       → denied: 23514 new row for relation "page_views" violates check constraint "page_views_page
  🔒 cannot read sentence_embeddings
       → denied: 42501 permission denied for table sentence_embeddings
  🔒 cannot call match_sentences()
       → denied: 42501 permission denied for function match_sentences
  🔒 cannot insert into sentence_embeddings
       → denied: 42501 permission denied for table sentence_embeddings

🔑 Logged-in user (rls-test-a@example.com, anon key + user JWT)
  ✅ can read sentence_embeddings
       → ok (5 rows)
  ✅ can call match_sentences()
       → ok (5 rows)
  ✅ match_sentences() caps match_count at 50
       → ok (50)
```

```
🔒 cannot insert into sentence_embeddings
     → denied: 42501 permission denied for table sentence_embeddings
🔒 cannot delete from sentence_embeddings
     → denied: 42501 permission denied for table sentence_embeddings
🔒 cannot read page_views rows
     → denied: 42501 permission denied for table page_views

📔 question_history: rls-test-a@example.com (A) vs rls-test-b@example.com (B)
✅ A can read A's own row
     → ok (1 rows)
🔒 B cannot read A's row (even by exact id)
     → 0 rows returned
🔒 B listing all history sees none of A's rows
     → 0 rows returned
🔒 B cannot delete A's row (0 rows affected)
     → 0 rows returned
✅ …and A's row still exists afterwards
     → ok (1 rows)
🔒 B cannot update A's row (no UPDATE grant/policy)
     → denied: 42501 permission denied for table question_history
🔒 A cannot insert a row owned by B (user_id not insertable)
     → denied: 42501 permission denied for table question_history
🔒 A cannot backdate created_at
     → denied: 42501 permission denied for table question_history
🔒 anonymous visitor cannot read any history
     → denied: 42501 permission denied for table question_history
✅ A can delete own row
     → ok (1 rows)

🗄️ Server-only tables (Edge Functions via service_role only)
🔒 user cannot read question_embeddings_cache
     → denied: 42501 permission denied for table question_embeddings_cache
🔒 user cannot poison question_embeddings_cache
     → denied: 42501 permission denied for table question_embeddings_cache
🔒 user cannot read api_usage (rate-limit log)
     → denied: 42501 permission denied for table api_usage
🔒 user cannot delete api_usage rows to reset their limit
     → denied: 42501 permission denied for table api_usage
🔒 user cannot call consume_rate_limit() (would burn others' quota)
     → denied: 42501 permission denied for function consume_rate_limit

============================================================
29/29 checks passed
============================================================
```

## Appendix B: Edge Function test (`python scripts/test_functions.py`)

```
============================================================
Edge Function Security Test
```

```
============================================================

🧠 embed-question
  ✅ no token → 401
       → HTTP 401 {"error":"Missing Authorization header"}
  ✅ anon key as token (no user) → 401
       → HTTP 401 {"error":"Not signed in"}
  ✅ signed-in user → 200 with 1536-dim embedding
       → HTTP 200 (dims=1536)
  ✅ empty question → 400
       → HTTP 400 {"error":"question must be a string of 1-1000 characters"}
  ✅ question > 1000 chars → 400
       → HTTP 400 {"error":"question must be a string of 1-1000 characters"}
  ✅ non-string question → 400
       → HTTP 400 {"error":"question must be a string of 1-1000 characters"}
  ✅ CORS preflight from evil.example.com is not allowed
       → Access-Control-Allow-Origin: https://trevorwattles.github.io

🤖 generate-answer
  ✅ no token → 401
       → HTTP 401 {"error":"Missing Authorization header"}
  ✅ empty context_talks → 400 (readiness probe, no GPT call)
       → HTTP 400 {"error":"context_talks must be a non-empty array"}
  ✅ 6 talks (> 5) → 400
       → HTTP 400 {"error":"context_talks may have at most 5 talks"}
  ✅ context over 60k chars → 400
       → HTTP 400 {"error":"context_talks total must be under 60000 characters"}
  ✅ talk missing text → 400
       → HTTP 400 {"error":"each talk needs string title and text"}
  ✅ grounded question → 200, cites the talk
       → HTTP 200
         "The talk teaches that prayer is a way to communicate with our Heavenly Father. It advises
praying both in the morning and at night and emphasizes listening for …"
  ✅ off-topic question → 200, should decline (read it!)
       → HTTP 200
         "The excerpts provided do not contain information about the capital of France.…"

🚀 Custom feature: cache, history, rate limit
  ✅ new question → cache miss (calls OpenAI)
       → HTTP 200 (cached=False)
  ✅ same question, different case/spacing → cache hit ($0)
       → HTTP 200 (cached=True)
  ✅ generate-answer saves history and returns history_id
       → HTTP 200 (history_id=True)
  ✅ history row owned by caller; javascript: URL stripped
       → owner matches=True, stored url=''
  ✅ 31st answer this hour → 429 (no OpenAI call)
       → HTTP 429 Limit of 30 answers per hour reached. Try again later.

============================================================
```

```
19/19 checks passed
============================================================
```

## Appendix C: Search query plans (`python scripts/explain_search.py`)

After indexes. Before-index numbers are in `ARCHITECTURE.md`: keyword worst case 91 ms (seq scan of all 26,256 rows), semantic 17–23 s (exact scan).

```
🔍 ILIKE '%faith%'  →  1.72 ms
     Limit (rows=20, removed_by_filter=0)
       Bitmap Heap Scan (rows=20, removed_by_filter=0)
         Bitmap Index Scan using sentence_embeddings_text_trgm_idx (rows=1565, removed_by_filter=0)

🔍 ILIKE '%Kirtland%'  →  0.79 ms
     Limit (rows=20, removed_by_filter=0)
       Bitmap Heap Scan (rows=20, removed_by_filter=0)
         Bitmap Index Scan using sentence_embeddings_text_trgm_idx (rows=47, removed_by_filter=0)

🔍 ILIKE '%xylophone%'  →  0.33 ms
     Limit (rows=0, removed_by_filter=0)
       Bitmap Heap Scan (rows=0, removed_by_filter=0)
         Bitmap Index Scan using sentence_embeddings_text_trgm_idx (rows=0, removed_by_filter=0)

🧠 nearest 20 to a sentence about 'peace'  →  3.47 ms warm (694 ms cold)  (recall vs exact: 100%)
     Limit (rows=20, removed_by_filter=0)
       Index Scan using sentence_embeddings_embedding_hnsw_idx (rows=20, removed_by_filter=0)

🧠 nearest 20 to a sentence about 'prayer'  →  4.42 ms warm (1260 ms cold)  (recall vs exact: 90%)
     Limit (rows=20, removed_by_filter=0)
       Index Scan using sentence_embeddings_embedding_hnsw_idx (rows=20, removed_by_filter=0)

🧠 nearest 20 to a sentence about 'temple'  →  5.00 ms warm (800 ms cold)  (recall vs exact: 100%)
     Limit (rows=20, removed_by_filter=0)
       Index Scan using sentence_embeddings_embedding_hnsw_idx (rows=20, removed_by_filter=0)
```